

Préparation au Concours National Commun 2026 — Informatique MP

Épreuve d'Informatique — Corrigé

Filière MP — Durée : 4 heures — Quatre problèmes indépendants

*Autour de la blockchain Ethereum : algorithmique, arbres de Merkle, partition (programmation dynamique) et finance décentralisée*Auteur : **Pr Agrégé El Hadiq Zouhair**cpgeacademy.org — Tous droits réservés © cpgeacademy.org

Avis important. Ce document est une **initiative personnelle** du professeur, à but strictement pédagogique. Il ne s'agit **pas** d'un sujet officiel du Concours National Commun ; il s'en inspire uniquement par la forme et l'esprit. **Consignes.** L'épreuve comporte **quatre problèmes indépendants**. Les questions sont numérotées **en continu** sur l'ensemble du sujet ; le corrigé reprend la même numérotation. On pourra traiter les problèmes dans l'ordre de son choix.

Problème I — Algorithmique de la blockchain Ethereum

Partie I — Unités et compteur de gaz

Question 1. On multiplie par 10^{18} .

```

1 def ether_vers_wei(e):
2     return e * 10**18

```

Complexité $O(1)$ opérations sur grands entiers.**Question 2.** On parcourt les coûts en maintenant le gaz disponible.

```

1 def gaz_restant(plafond, couts):
2     restant = plafond
3     for c in couts:
4         if restant < c:
5             return "panne de gaz"
6         restant -= c
7     return restant

```

La boucle est finie (**len**(couts) tours), d'où la terminaison ; complexité $O(m)$ avec $m = \text{len}(\text{couts})$.**Question 3.** C'est une série géométrique de raison $1/2 \in]0, 1[$, donc convergente :

$$C = \sum_{i=0}^{+\infty} g \left(\frac{1}{2}\right)^i = \frac{g}{1 - \frac{1}{2}} = 2g.$$

Une exécution qui s'arrête après m itérations consomme $\sum_{i=0}^{m-1} g/2^i = g \frac{1 - (1/2)^m}{1 - 1/2} = 2g(1 - 2^{-m}) < 2g$. Le coût cumulé est donc *toujours* strictement majoré par $2g$, indépendamment de m . $\square$

Partie II — Encodage hexadécimal des adresses

Question 4.

```

1 def entier_vers_hex(n):
2     if n == 0:
3         return "0"
4     chiffres = "0123456789abcdef"
5     s = ""
6     while n > 0:
7         s = chiffres[n % 16] + s
8         n //= 16
9     return s

```

Question 5.

```

1 def hex_vers_entier(s):
2     chiffres = "0123456789abcdef"
3     n = 0
4     for c in s:
5         n = n * 16 + chiffres.index(c)
6     return n

```

Question 6. Soit $n \geq 1$ et d le nombre de chiffres produits. À chaque tour, n est remplacé par $\lfloor n/16 \rfloor$; la boucle s'arrête dès que n atteint 0. Le nombre de tours est donc le plus petit d tel que $\lfloor n/16^d \rfloor = 0$, c'est-à-dire $16^d > n$, soit $d > \log_{16} n$. Le plus petit tel entier est $d = \lfloor \log_{16} n \rfloor + 1$. Chaque tour coûte $O(1)$ (hors coût des grands entiers), donc la complexité est $\Theta(d) = \Theta(\log n)$ tours.

Question 7. Terminaison. La quantité entière $n \geq 0$ décroît strictement à chaque tour (on passe de n à $\lfloor n/16 \rfloor < n$ tant que $n \geq 1$) et reste positive : c'est un *variant de boucle*, donc la boucle termine.

Correction. Notons $c_{d-1} \dots c_1 c_0$ les chiffres produits (poids faible c_0). Par construction, à chaque tour on extrait $c_j = n_j \bmod 16$ puis $n_{j+1} = \lfloor n_j/16 \rfloor$, avec $n_0 = n$. Par la division euclidienne, $n_j = 16 n_{j+1} + c_j$ avec $0 \leq c_j < 16$. Une récurrence immédiate donne $n = \sum_{j=0}^{d-1} c_j 16^j$. Or `hex_vers_entier` évalue par schéma de Horner exactement $\sum_{j=0}^{d-1} c_j 16^j$. Donc `hex_vers_entier(entier_vers_hex(n))` est égal à n . Le cas $n = 0$ est traité à part et renvoie "0", décodé en 0. $\square$

Vérification numérique : $255 \mapsto \text{"ff"}$, $16 \mapsto \text{"10"}$, $7271972807 \mapsto \text{"1b1717fc7"}$ (9 chiffres, et $\lfloor \log_{16} 7271972807 \rfloor + 1 = 9$).

Partie III — Arbres de Merkle

Question 8.

```

1 def parent(a, b):
2     return (31 * a + b) % 1000003
3
4 def racine_merkle(feuilles):
5     niveau = list(feuilles)
6     while len(niveau) > 1:
7         if len(niveau) % 2 == 1:
8             niveau.append(niveau[-1])
9             niveau = [parent(niveau[i], niveau[i+1])
10                      for i in range(0, len(niveau), 2)]
11     return niveau[0]

```

Vérification : niveau 0 = [1, 2, 3, 4] ; $\text{parent}(1, 2) = 33$, $\text{parent}(3, 4) = 97$; niveau 1 = [33, 97] ; $\text{parent}(33, 97) = 33 \cdot 31 + 97 = 1120$. Donc la racine vaut **1120**. (De même $\text{racine_merkle}([1, 2, 3]) = 1119$ après duplication du 3.)

Question 9. À chaque niveau, si la liste a une taille t , on effectue $\lceil t/2 \rceil$ appels à parent et la taille suivante est $\lceil t/2 \rceil$. Partant de $t_0 = n$, on a $t_{k+1} = \lceil t_k/2 \rceil \leq t_k/2 + 1$. Le nombre total d'appels est $\sum_{k \geq 0} \lceil t_k/2 \rceil = \sum_{k \geq 1} t_k \leq \sum_{k \geq 1} (n/2^k + 2)$. La partie géométrique se majore par n , et il y a $O(\log n)$ niveaux, donc le total est $\leq n + O(\log n) = O(n)$. $\square$

Question 10.

```

1 def preuve_inclusion(feuilles, i):
2     niveau = list(feuilles)
3     preuve = []
4     idx = i
5     while len(niveau) > 1:
6         if len(niveau) % 2 == 1:
7             niveau.append(niveau[-1])
8         if idx % 2 == 0:
9             preuve.append((niveau[idx+1], 'R'))
10        else:
11            preuve.append((niveau[idx-1], 'L'))
12        niveau = [parent(niveau[j], niveau[j+1])
13                  for j in range(0, len(niveau), 2)]
14        idx //= 2
15    return preuve
16
17 def verifier(feuille, preuve, racine):
18     acc = feuille
19     for frere, cote in preuve:
20         if cote == 'R':
21             acc = parent(acc, frere)
22         else:
23             acc = parent(frere, acc)
24    return acc == racine

```

Vérification : $\text{preuve_inclusion}([1, 2, 3, 4], 0) = [(2, 'R'), (97, 'R')]$; puis $\text{parent}(1, 2) = 33$, $\text{parent}(33, 97) = 1120 = \text{racine}$.

Question 11. Récurrence sur la hauteur h . Pour $h = 0$ (une seule feuille), la racine est la feuille ; une preuve vide n'est acceptée que si $\text{feuille} = \text{racine}$. Supposons la propriété vraie à la hauteur h et considérons un arbre de hauteur $h + 1$. La vérification recalcule d'abord $r' = \text{parent}(\text{feuille}, \text{frère})$ (ou l'inverse), puis vérifie que r' , muni du reste de la preuve, mène à la racine : c'est une preuve d'inclusion de r' dans un arbre de hauteur h , valide par hypothèse de récurrence, donc r' est bien un nœud interne du niveau h . Comme parent est résistante aux collisions, l'unique antécédent connu de r' est le couple $(\text{feuille}, \text{frère})$ réellement présent ; la feuille figure donc parmi celles ayant produit la racine. $\square$

La preuve contient un couple par niveau, soit une taille $O(\log n)$: c'est l'intérêt majeur des arbres de Merkle (certificat logarithmique).

Partie IV — Sélection aléatoire d'un validateur

Question 12. Le premier succès arrive au k -ième crénneau si les $k - 1$ premiers sont des échecs (probabilité $(1 - p)^{k-1}$) suivis d'un succès (probabilité p) :

$$\mathbb{P}(N = k) = (1 - p)^{k-1} p, \quad k \geq 1.$$

C'est la *loi géométrique* de paramètre p , d'espérance $\mathbb{E}[N] = \sum_{k \geq 1} k(1 - p)^{k-1} p = \frac{1}{p}$.

Question 13. $\mathbb{P}(N > k)$ est la probabilité de k échecs consécutifs, soit $(1 - p)^k$. Comme $0 < 1 - p < 1$, $(1 - p)^k \rightarrow 0$ quand $k \rightarrow +\infty$, donc $\mathbb{P}(N < +\infty) = 1 - \lim_k \mathbb{P}(N > k) = 1$: terminaison presque sûre. $\square$

Question 14. Pour $p = 1/4$: $\mathbb{E}[N] = 4$, et $\mathbb{P}(N \leq 4) = 1 - (3/4)^4 = 1 - \frac{81}{256} = \frac{175}{256} \approx 0,684$.

Partie V — Partition équilibrée par programmation dynamique

Question 15. Si T est impaire, aucune part entière ne peut valoir $T/2$ (qui n'est pas entier) ; les deux parts, de sommes entières égales, donneraient T pair. Donc la réponse est *non*. $\square$

Question 16. Un *certificat* est la liste des indices d'un sous-ensemble candidat. La vérification calcule sa somme et teste l'égalité à C , en temps $O(n)$ (polynomial). Le problème est donc dans **NP**. (Il s'agit du problème SUBSET-SUM, NP-complet.)

Question 17. Un sous-ensemble des i premiers éléments de somme c ou bien n'utilise pas le i -ième élément (donc c'est un sous-ensemble des $i - 1$ premiers de somme c), ou bien l'utilise (donc c'est un sous-ensemble des $i - 1$ premiers de somme $c - S[i - 1]$, licite si $c \geq S[i - 1]$) :

$$A[i][c] = A[i - 1][c] \vee (c \geq S[i - 1] \wedge A[i - 1][c - S[i - 1]]),$$

avec l'initialisation $A[0][0] = \text{vrai}$ et $A[0][c] = \text{faux}$ pour $c \geq 1$.

Question 18.

```

1 def peut_partitionner(S):
2     T = sum(S)
3     if T % 2 == 1:
4         return False
5     C = T // 2
6     dp = [False] * (C + 1)
7     dp[0] = True
8     for x in S: # traite un element a la fois
9         for c in range(C, x - 1, -1): # parcours DECROISSANT
10             if dp[c - x]:
11                 dp[c] = True
12     return dp[C]
```

Le parcours *décroissant* de c garantit que $dp[c - x]$ se réfère encore à l'état *avant* l'introduction de x : on n'utilise donc jamais deux fois le même élément. Complexité : $O(nC)$ en temps et $O(C)$ en espace. Comme C dépend de la *valeur* des données (et non seulement de leur nombre), la taille de C est exponentielle en le nombre de bits de codage : l'algorithme est dit *pseudo-polynomial*.

Question 19. Invariant. Après le traitement des i premiers éléments, pour tout $0 \leq c \leq C$, $dp[c]$ est vrai si et seulement s'il existe un sous-ensemble des i premiers éléments de somme c (c'est-à-dire $dp[c] = A[i][c]$).

Initialisation ($i = 0$) : seul $dp[0]$ est vrai, ce qui correspond au sous-ensemble vide ; conforme à $A[0]$. *Hérédité* : supposons l'invariant vrai après $i - 1$ éléments et traitons $x = S[i - 1]$. Pour chaque c de C à x , on met $dp[c]$ à vrai lorsque $dp[c - x]$ est vrai. Le parcours décroissant assure que $dp[c - x]$ vaut encore $A[i - 1][c - x]$; combiné à la valeur préexistante $A[i - 1][c]$, on obtient exactement $A[i][c] = A[i - 1][c] \vee (c \geq x \wedge A[i - 1][c - x])$. L'invariant est donc vrai après i éléments.

Conclusion : après les n éléments, $dp[C] = A[n][C]$, qui est vrai ssi une partition équilibrée existe.

Terminaison : les deux boucles **for** sont finies. $\square$

Vérifications numériques : `peut_partitionner([1, 1, 5, 7]) = True` (part $[1, 1, 5]$ de somme 7), `peut_partitionner([1, 1, 5, 8]) = False` (somme 15 impaire). Sur 2000 jeux aléatoires, `peut_partitionner` coïncide avec une recherche exhaustive.

Problème II — Arbres de Merkle et preuves d'inclusion

Partie I — La fonction de combinaison

Question 20.

```

1 def parent(a, b):
2     return (31 * a + b) % 1000003

```

Question 21. $\text{parent}(1, 2) = 31 \cdot 1 + 2 = 33$; $\text{parent}(3, 4) = 31 \cdot 3 + 4 = 97$; $\text{parent}(33, 97) = 31 \cdot 33 + 97 = 1023 + 97 = 1120$.

Question 22. Le calcul de la racine doit donner *le même* résultat sur tous les nœuds du réseau pour que le consensus soit possible. Si `parent` pouvait renvoyer des valeurs différentes pour les mêmes entrées (aléa, dépendance à l'environnement), deux nœuds honnêtes partant des mêmes feuilles obtiendraient des racines différentes et ne pourraient pas s'accorder. Le déterminisme garantit la *reproductibilité*.

Question 23. L'ensemble de départ $\mathbb{N}^2$ est infini (ou, en pratique, de cardinal bien plus grand que p), tandis que l'ensemble d'arrivée $\{0, \dots, p-1\}$ est de cardinal fini p . Par le principe des tiroirs, h ne peut être injective : des collisions *existent* nécessairement. La résistance aux collisions n'affirme donc pas leur absence, mais qu'il est *calculatoirement difficile* d'en exhiber une : c'est une hypothèse de sécurité, pas un théorème d'injectivité. $\square$

Partie II — Construction de la racine

Question 24.

```

1 def racine_merkle(feuilles):
2     niveau = list(feuilles)
3     while len(niveau) > 1:
4         if len(niveau) % 2 == 1:
5             niveau.append(niveau[-1]) # duplique le dernier
6             niveau = [parent(niveau[i], niveau[i+1])
7                       for i in range(0, len(niveau), 2)]
8     return niveau[0]

```

Question 25. Niveau 0 = $[1, 2, 3, 4]$; $\text{parent}(1, 2) = 33$, $\text{parent}(3, 4) = 97$, donc niveau 1 = $[33, 97]$; $\text{parent}(33, 97) = 1120$. Racine = **1120**.

Question 26. Niveau 0 = $[1, 2, 3]$, impair $\Rightarrow$ on duplique : $[1, 2, 3, 3]$. $\text{parent}(1, 2) = 33$, $\text{parent}(3, 3) = 31 \cdot 3 + 3 = 96$, donc niveau 1 = $[33, 96]$; $\text{parent}(33, 96) = 1023 + 96 = 1119$. Racine = **1119**.

Question 27. On a $t_{k+1} = \lceil t_k/2 \rceil \leq t_k/2 + 1$. Par récurrence $t_k \leq n/2^k + 2$, donc $t_k \leq 2$ dès que $n/2^k \leq 1$, c'est-à-dire $k \geq \log_2 n$: il y a $O(\log n)$ niveaux. Le nombre d'appels à `parent` au niveau k est $\lceil t_k/2 \rceil$, et le total vaut $\sum_{k \geq 1} t_k \leq \sum_{k \geq 1} (n/2^k + 2) \leq n + O(\log n) = O(n)$. $\square$

Partie III — Preuves d'inclusion

Question 28.

```

1 def preuve_inclusion(feuilles, i):
2     niveau = list(feuilles)
3     preuve = []
4     idx = i
5     while len(niveau) > 1:
6         if len(niveau) % 2 == 1:

```

```

7     niveau.append(niveau[-1])
8     if idx % 2 == 0:
9         preuve.append((niveau[idx+1], 'R')) # frere a droite
10    else:
11        preuve.append((niveau[idx-1], 'L')) # frere a gauche
12    niveau = [parent(niveau[j], niveau[j+1])
13              for j in range(0, len(niveau), 2)]
14    idx //= 2
15    return preuve

```

Question 29.

```

1 def verifier(feuille, preuve, racine):
2     acc = feuille
3     for frere, cote in preuve:
4         if cote == 'R':
5             acc = parent(acc, frere)
6         else:
7             acc = parent(frere, acc)
8     return acc == racine

```

Exemple : `preuve_inclusion([1, 2, 3, 4], 0) = [(2, 'R'), (97, 'R')]`. La vérification calcule `parent(1, 2) = 33` puis `parent(33, 97) = 1120`, égal à la racine : `True`.

Question 30. La preuve contient exactement un couple par niveau au-dessus de la feuille, soit $H(n) = \lceil \log_2 n \rceil = O(\log n)$ couples. On transmet donc $O(\log n)$ valeurs au lieu des n feuilles : pour $n = 10^6$, une vingtaine de valeurs suffisent. C'est ce qui rend les preuves de Merkle utilisables sur une blockchain, où l'espace de stockage on-chain est coûteux.

Question 31. Récurrence sur h . Base $h = 0$: l'arbre est réduit à une feuille, la racine est cette feuille, et la preuve est vide ; `verifier` renvoie `True` seulement si `feuille = racine`, donc la feuille est bien celle ayant produit la racine. *Hérédité* : supposons le résultat vrai pour tout arbre de hauteur h , et soit un arbre de hauteur $h + 1$. La vérification calcule d'abord $r' = \text{parent}(\text{feuille}, f)$ (ou `parent(f, feuille)`), puis vérifie que r' , muni du reste de la preuve, mène à la racine : c'est une preuve d'inclusion valide de r' dans l'arbre des nœuds de niveau 1, de hauteur h . Par hypothèse de récurrence, r' est bien un nœud de ce niveau, produit par un couple réel de l'arbre. Comme `parent` résiste aux collisions, le seul antécédent connu de r' est le couple effectivement présent : c'est `(feuille, f)`. Donc `feuille` figure parmi les feuilles ayant produit la racine. $\square$

Partie IV — Hauteur et mise à jour

Question 32. À chaque niveau, la taille passe de t à $\lceil t/2 \rceil$; partant de n , il faut le plus petit H tel que $\lceil \dots \lceil n/2 \rceil \dots \rceil = 1$ après H divisions, soit $2^{H-1} < n \leq 2^H$ pour $n \geq 2$, d'où $H(n) = \lceil \log_2 n \rceil$ (et $H(1) = 0$). On obtient

$$H(1) = 0, H(2) = 1, H(3) = 2, H(4) = 2, H(5) = 3, H(8) = 3, H(9) = 4.$$

Question 33. Seuls les nœuds situés *sur le chemin* de la feuille modifiée jusqu'à la racine changent de valeur : à chaque niveau, un seul nœud dépend de la feuille modifiée (son ancêtre). Il y a $H(n) = O(\log n)$ tels nœuds ; recalculer chacun coûte un appel à `parent`, d'où une mise à jour en $O(\log n)$ appels. Une implémentation simple, correcte mais non optimale, recalcule toute la racine :

```

1 def maj_racine(feUILLES, i, v):
2     nouvelles = list(feUILLES)
3     nouvelles[i] = v
4     return racine_merkle(nouvelles)

```

Cette version coûte $O(n)$; l'argument ci-dessus montre qu'une version conservant les nœuds inchangés atteindrait $O(\log n)$. $\square$

Vérifications numériques (exécutées en Python) : `racine_merkle([1, 2, 3, 4]) = 1120`, `racine_merkle([1, 2, 3]) = 1119`, et les hauteurs annoncées correspondent à $\lceil \log_2 n \rceil$.

Problème III — Le problème de partition et la programmation dynamique

Partie I — Recherche exhaustive

Question 34.

```

1 def demi_somme(S):
2     T = sum(S)
3     if T % 2 == 1:
4         return None
5     return T // 2

```

Question 35.

```

1 import itertools
2
3 def partition_exhaustive(S):
4     T = sum(S)
5     if T % 2 == 1:
6         return False
7     C = T // 2
8     for r in range(len(S) + 1):
9         for comb in itertools.combinations(S, r):
10             if sum(comb) == C:
11                 return True
12     return False

```

Question 36. Un ensemble à n éléments possède 2^n sous-ensembles (chaque élément est pris ou non). La boucle les parcourt tous dans le pire des cas (instance négative), et chaque test de somme coûte $O(n)$: la complexité est $\Theta(n2^n) = \Theta(2^n)$ à un facteur polynomial près. Pour $n = 60$, $2^{60} \approx 10^{18}$: irréalisable. $\square$

Partie II — Le problème de décision Subset-Sum

Question 37. Le certificat est la *liste des indices* d'un sous-ensemble candidat $I \subseteq \{0, \dots, n-1\}$. La vérification calcule $\sum_{k \in I} S[k]$ et teste l'égalité à C ; elle lit chaque indice une fois, d'où une complexité $O(n)$, polynomiale. Le certificat a une taille $O(n)$, elle aussi polynomiale.

Question 38. Puisque toute instance positive admet un certificat de taille polynomiale vérifiable en temps polynomiale, Subset-Sum est dans **NP**. (On admet qu'il est de plus NP-complet, donc parmi les plus difficiles de NP.) $\square$

Partie III — Récurrence et table

Question 39. Un sous-ensemble des i premiers éléments de somme c soit *n'utilise pas* $S[i-1]$ (c'est alors un sous-ensemble des $i-1$ premiers de somme c , d'où $A[i-1][c]$), soit *l'utilise* (c'est alors un sous-ensemble des $i-1$ premiers de somme $c - S[i-1]$, licite seulement si $c \geq S[i-1]$, d'où $A[i-1][c - S[i-1]]$). D'où $A[i][c] = A[i-1][c] \vee (c \geq S[i-1] \wedge A[i-1][c - S[i-1]])$. Initialisation : avec zéro élément, seule la somme 0 est atteignable, donc $A[0][0] = \text{vrai}$ et $A[0][c] = \text{faux}$ pour $c \geq 1$. $\square$

Question 40.

```

1 def subset_sum_2d(S, C):
2     n = len(S)
3     A = [[False] * (C + 1) for _ in range(n + 1)]
4     for i in range(n + 1):

```



```

5     A[i][0] = True # somme 0 : sous-ensemble vide
6     for i in range(1, n + 1):
7         x = S[i - 1]
8         for c in range(C + 1):
9             A[i][c] = A[i-1][c] or (c >= x and A[i-1][c - x])
10    return A[n][C]

```

Complexité : $O(nC)$ en temps (deux boucles imbriquées) et $O(nC)$ en espace (table complète).

Partie IV — Version optimisée en espace

Question 41.

```

1 def peut_partitionner(S):
2     T = sum(S)
3     if T % 2 == 1:
4         return False
5     C = T // 2
6     dp = [False] * (C + 1)
7     dp[0] = True
8     for x in S:
9         for c in range(C, x - 1, -1): # DECROISSANT
10            if dp[c - x]:
11                dp[c] = True
12    return dp[C]

```

Question 42. Le tableau dp représente la ligne courante $A[i-1][\cdot]$ que l'on met à jour en $A[i][\cdot]$ « sur place ». En parcourant c de façon *décroissante*, la case $dp[c - x]$ lue n'a pas encore été modifiée à ce tour : elle vaut encore $A[i-1][c - x]$, ce qui est correct. Avec un parcours *croissant*, $dp[c - x]$ aurait déjà pu passer à *True* *au même tour*, ce qui reviendrait à utiliser x **plusieurs fois** (comme dans un sac à dos avec répétitions) : on résoudrait un autre problème et la réponse serait fausse.

Question 43. Complexité : $O(nC)$ en temps, $O(C)$ en espace. L'entrée se code sur $O(\sum_i \log S_i)$ bits ; or C peut être de l'ordre de $\max_i S_i$, donc *exponentiel* en le nombre de bits de codage. L'algorithme est polynomial en la *valeur* C mais non en la *taille* de l'entrée : on dit qu'il est *pseudo-polynomial*. $\square$

Partie V — Reconstruction et correction

Question 44.

```

1 def sous_ensemble(S, C):
2     n = len(S)
3     A = [[False] * (C + 1) for _ in range(n + 1)]
4     for i in range(n + 1):
5         A[i][0] = True
6     for i in range(1, n + 1):
7         x = S[i - 1]
8         for c in range(C + 1):
9             A[i][c] = A[i-1][c] or (c >= x and A[i-1][c - x])
10    if not A[n][C]:
11        return None
12    res, c = [], C
13    for i in range(n, 0, -1):
14        if not A[i-1][c]: # S[i-1] est indispensable
15            res.append(S[i-1])
16            c -= S[i-1]
17    return res[::-1]

```

Vérification : `sous_ensemble`([1, 1, 5, 7], 7) = [1, 1, 5] (somme 7).

Question 45. Invariant. Après le traitement des i premiers éléments de S dans `peut_partitionner`, pour tout $0 \leq c \leq C$, `dp[c]` est vrai si et seulement s'il existe un sous-ensemble des i premiers éléments de somme c (c.-à-d. `dp[c] = A[i][c]`).

Base ($i = 0$) : seul `dp[0]` est vrai, correspondant au sous-ensemble vide : conforme à `A[0]`.

Hérédité : on suppose `dp = A[i-1][·]` et on traite $x = S[i-1]$. Pour chaque c (de C à x), on met `dp[c]` à `True` si `dp[c-x]` est vrai. Le parcours décroissant garantit `dp[c-x] = A[i-1][c-x]` ; combiné à la valeur préexistante `A[i-1][c]` de `dp[c]`, on obtient exactement `A[i][c] = A[i-1][c] ∨ (c ≥ x ∧ A[i-1][c-x])`. L'invariant est donc préservé. *Conclusion* : après les n éléments, `dp[C] = A[n][C]`, vrai ssi une partition équilibrée existe : la fonction est correcte. **Terminaison** : toutes les boucles `for` portent sur des intervalles finis. □

Vérifications numériques (Python) : sur 2000 listes aléatoires, `peut_partitionner` et `partition_exhaustive` renvoient *toujours* le même résultat. Exemples : [1, 1, 5, 7] ↦ `True`, [1, 1, 5, 8] ↦ `False`, [3, 3, 3, 3] ↦ `True`.

Problème IV — Finance décentralisée : teneurs de marché automatisés

Partie I — Prix et produit constant

Question 46.

```
1 def prix(x, y):
2     return y / x
```

Pour $x = 10$, $y = 100$: $\text{prix}(10, 100) = 100/10 = 10$ USDC par ETH.

Question 47. La conservation du produit s'écrit $(x + \Delta x)(y - \Delta y) = xy$, d'où $y - \Delta y = \frac{xy}{x + \Delta x}$, puis

$$\Delta y = y - \frac{xy}{x + \Delta x} = \frac{y(x + \Delta x) - xy}{x + \Delta x} = \frac{y \Delta x}{x + \Delta x}. \quad \square$$

Question 48.

```
1 def sortie_amm(x, y, dx):
2     return (y * dx) / (x + dx)
```

$$\text{sortie_amm}(100, 100, 100) = \frac{100 \cdot 100}{200} = 50.$$

Partie II — Bornes et monotonie

Question 49. Pour $\Delta x > 0$ et $x > 0$, on a $\frac{\Delta x}{x + \Delta x} < 1$, donc $\Delta y = y \cdot \frac{\Delta x}{x + \Delta x} < y$. La réserve de Y ne peut donc jamais être entièrement vidée. $\square$

Question 50. Écrivons $\Delta y(\Delta x) = y \left(1 - \frac{x}{x + \Delta x}\right)$. La fonction $\Delta x \mapsto \frac{x}{x + \Delta x}$ est strictement décroissante (dérivée $-x/(x + \Delta x)^2 < 0$), donc Δy est strictement *croissante*. De plus $\frac{x}{x + \Delta x} \rightarrow 0$ quand $\Delta x \rightarrow +\infty$, donc $\Delta y \rightarrow y$ sans jamais l'atteindre. Plus l'échange est gros, plus Δy approche son plafond y : le rendement marginal s'effondre, ce qui traduit un *glissement* croissant. $\square$

Question 51. Le prix effectif est

$$\frac{\Delta x}{\Delta y} = \frac{\Delta x(x + \Delta x)}{y \Delta x} = \frac{x + \Delta x}{y} = \frac{x}{y} + \frac{\Delta x}{y} > \frac{x}{y}.$$

Il dépasse strictement le prix marginal x/y dès que $\Delta x > 0$, et tend vers x/y quand $\Delta x \rightarrow 0^+$. Ainsi tout achat s'effectue à un prix *strictement moins favorable* que le prix marginal, et ce d'autant plus que l'échange est gros : la courbe (une hyperbole) protège la réserve d'un drainage complet. $\square$

Partie III — Frais cumulés et série géométrique

Question 52. C'est une série géométrique de raison $\rho \in]0, 1[$, donc convergente :

$$\Phi = f v_0 \sum_{i=0}^{+\infty} \rho^i = \frac{f v_0}{1 - \rho}. \quad \square$$

Question 53. Pour $f = 0,003$, $v_0 = 1000$, $\rho = 0,9$: $\Phi = \frac{0,003 \times 1000}{1 - 0,9} = \frac{3}{0,1} = 30$.

Question 54.

```

1 def frais_cumules(f, v0, rho, n):
2     total = 0.0
3     puissance = 1.0 # rho**0
4     for _ in range(n):
5         total += f * v0 * puissance
6         puissance *= rho
7     return total

```

L'erreur est le reste de la série :

$$\Phi - \text{frais_cumules}(f, v_0, \rho, n) = \sum_{i=n}^{+\infty} f v_0 \rho^i = \frac{f v_0 \rho^n}{1 - \rho},$$

qui tend vers 0 géométriquement en ρ^n . $\square$

Partie IV — Prêt sur-collatéralisé et intérêts composés

Question 55.

```

1 def collateral_requis(P, tau):
2     return P * tau // 100

```

`collateral_requis(100, 150) = 100 · 150 // 100 = 150.`

Question 56.

```

1 def dette(P, r, n):
2     return P * (1 + r)**n

```

La dette $P(1+r)^n$ est strictement croissante en n (car $1+r > 1$). Elle dépasse G ssi $(1+r)^n > G/P$, soit $n \ln(1+r) > \ln(G/P)$, c'est-à-dire $n > \frac{\ln(G/P)}{\ln(1+r)}$. Le plus petit entier vérifiant cette inégalité stricte est $n^* = \left\lfloor \frac{\ln(G/P)}{\ln(1+r)} \right\rfloor + 1$ (pour $G > P$, le membre de droite est positif). $\square$

Question 57. Pour $P = 100$, $r = 0,10$, $G = 150$: $\frac{\ln(1,5)}{\ln(1,1)} \approx \frac{0,405}{0,0953} \approx 4,254$, donc $n^* = \lfloor 4,254 \rfloor + 1 = 5$. Vérification : `dette(100, 0.1, 4) = 100 · 1,14 ≈ 146,41 ≤ 150` et `dette(100, 0.1, 5) = 100 · 1,15 ≈ 161,05 > 150`, d'où l'encadrement annoncé.

Terminaison. Une boucle `while dette(P, r, n) <= G: n += 1` termine : la suite $n \mapsto \text{dette}(P, r, n)$ est strictement croissante et non majorée (elle tend vers $+\infty$), donc elle finit par dépasser G après un nombre fini d'itérations (au plus n^*). L'entier $\lceil G/P \rceil - (1+r)^n \dots$ plus simplement, $G - \text{dette}(P, r, n)$ décroît strictement et devient négatif : la condition de boucle finit par être fausse. $\square$

Vérifications numériques (Python) : `prix(10, 100) = 10` ; `sortie_amm(100, 100, 100) = 50` et $(x + \Delta x)(y - \Delta y) = xy = 20000$; les sorties `sortie_amm(100, 100, Δx)` pour $\Delta x \in \{1, 10, 100, 1000, 10^6\}$ valent approximativement 0,99; 9,09; 50; 90,91; 99,99 et croissent vers $y = 100$; $\Phi = 30$; $n^* = 5$ avec `dette(100, 0.1, 4) ≈ 146,41` et `dette(100, 0.1, 5) ≈ 161,05`.